

HANDS_ON_Abhaengigkeiten

Hands-on: Job-Abhängigkeiten in Apache NiFi

Apache NiFi 2.8.0, Single-User-Login (admin), UI unter <https://localhost:8443/nifi>

Vorbemerkung: Wie drückt NiFi “Abhängigkeit” aus?

NiFi ist kein klassischer Job-Scheduler wie Airflow oder cron. Es gibt keinen “Job A, dann Job B”-Knopf. Abhängigkeiten entstehen auf vier Wegen, die wir in den Übungen durchspielen:

Mechanismus	“B startet, wenn ...”	Übung
Datenabhängigkeit (Connection/Relationship)	... A eine FlowFile auf der Relationship erzeugt	Lab 1
Signal (Wait / Notify)	... A ein benanntes Signal gesetzt hat	Lab 2
Barriere / Fan-in (MergeContent, Wait-Counter)	... N Eingaben vorliegen	Lab 3
Zeit + Bedingung (Cron-Scheduling, RouteOnAttribute)	... ein Zeitpunkt erreicht ist bzw. eine Bedingung gilt	Lab 4

Kernidee: Ein Prozessor wird nur aktiv, wenn in seiner **eingehenden Queue** eine FlowFile liegt. “Job-Steuerung” heißt in NiFi: steuern, wann eine FlowFile in der Queue des nächsten Prozessors landet.

Gemeinsame Pfade (zum Kopieren)

```
Input:      /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/input
Output:     /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/output
Reports:    /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/reports
Archiv:     /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/archive
DB:         /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/data/db/fmea.db
Treiber:    /Users/grigo/gits/seminars/it-schulungen.com/Apache_NiFi/nifi-2.8.0/drivers/sqlite-jdbc-3.45.1.0.jar
```

Beispiel-CSV (immer wieder verwendbar):

```
cp ~/Knowledge/outputs/fmea_entities.csv <Input-Pfad>/
```

Lab 1: Datenabhängigkeit: “Erst laden, dann Erfolg melden”

Lernziel: Verstehen, dass die `success`-Relationship die einfachste Form einer Abhängigkeit ist: der nachgelagerte Prozessor läuft nur, wenn der vorige erfolgreich war.

Szenario: Eine CSV wird in die DB geladen. Nur bei Erfolg wird eine Marker-Datei `load_ok.txt` geschrieben; bei Fehler landet die Datei sichtbar im Output statt lautlos verworfen zu werden.

Flow:

```
GetFile —success—> PutDatabaseRecord —success—> PutFile (Marker)
                                   |
                                   —failure—> PutFile (Fehlerablage)
```

Schritte: 1. `GetFile` (aus dem vorhandenen Lab wiederverwenden), Input Directory = Input-Pfad. 2. `PutDatabaseRecord` (vorhanden): Record Reader = `CSVReader`, DBCP = `DBCPConnectionPool`, Statement Type = `INSERT`, Table Name = `fmea_entities`. 3. Neuer `PutFile` “Erfolg”: Directory = Reports-Pfad. Relationship `success/failure` terminieren. 4. Neuer `PutFile` “Fehler”: Directory = `Output/errors`. Relationships terminieren. 5. Verbinde `PutDatabaseRecord` mit `success` -> Erfolg-PutFile, mit `failure` -> Fehler-PutFile. 6. Alles starten, CSV in Input kopieren.

Verifikation:

```
ls -l <Reports-Pfad>/          # Marker-Datei vorhanden -> Job-Kette erfolgreich
sqlite3 <DB> "SELECT COUNT(*) FROM fmea_entities;"
```

Fehlerfall provozieren (Diskussion): Tabellennamen in `PutDatabaseRecord` auf `gibt_es_nicht` ändern, CSV erneut einwerfen. Die FlowFile geht jetzt über `failure`. -> Abhängigkeit ist gerichtet nach Erfolg.

Diskussionsfragen: - Warum ist “auto-terminate failure” in Produktion eine schlechte Idee? - Was passiert, wenn beide `PutFile` auf dasselbe Verzeichnis schreiben?

Lab 2: Wait / Notify: echte Job-Abhängigkeit über ein Signal

Lernziel: Zwei unabhängige Flows so koppeln, dass Job B erst nach Abschluss von Job A startet, ohne dass B von A direkt FlowFiles bekommt. Das ist das NiFi-Pendant zu “B depends on A”.

Szenario: - **Job A** lädt die FMEA-CSV in die DB und meldet danach das Signal `fmea-loaded`. - **Job B** soll einen Report (Zeilenzahl) erzeugen, darf aber **erst** laufen, wenn `fmea-loaded` da ist.

Benötigte Controller Services (einmalig anlegen, beide aktivieren):

Hinweis NiFi 2.x: Die Services heißen jetzt `MapCacheServer` und `MapCacheClientService` (früher `DistributedMapCacheServer` / `DistributedMapCacheClientService`). Im Add-Dialog nach `MapCache` filtern.

1. `MapCacheServer` (Defaults reichen, Port 4557) -> aktivieren.

2. MapCacheClientService: Server Hostname = localhost, Server Port = 4557 -> aktivieren.

Reihenfolge: erst Server aktivieren, dann Client.

Flow:

```
Job A: GetFile -> PutDatabaseRecord ->success-> Notify (Signal: fmea-loaded)
Job B: GenerateFlowFile -> Wait ->success-> ExecuteSQL -> PutFile (Report)
      L Wait wartet auf Signal: fmea-loaded
```

Notify konfigurieren (Job A): - Release Signal Identifier = fmea-loaded - Distributed Cache Service = der Client von oben - Relationship success terminieren.

Wait konfigurieren (Job B): - Release Signal Identifier = fmea-loaded - Distributed Cache Service = derselbe Client - Target Signal Count = 1 - Relationship success -> weiter zu ExecuteSQL; expired -> PutFile "timeout" oder terminieren; failure terminieren.

ExecuteSQL (Job B): Database Connection Pooling Service = DBCPConnectionPool, SQL select query = SELECT COUNT(*) AS anzahl FROM fmea_entities. Danach PutFile in Reports-Pfad.

GenerateFlowFile (Job B Trigger): Scheduling = 10 sec (erzeugt regelmäßig einen "Versuch").

Verifikation / Aha-Effekt: 1. Starte **nur Job B**. Beobachte: die FlowFiles stauen sich in der Queue vor ExecuteSQL, weil Wait blockt. Es passiert nichts -> B wartet auf A. 2. Starte jetzt **Job A** und wirf die CSV ein. Sobald Notify feuert, lässt Wait die nächste FlowFile durch -> der Report wird geschrieben.

```
ls -l <Reports-Pfad>/
```

Praxis-Hinweise (im Live-Test verifiziert): 1. wait-Relationship als Self-Loop:

Die wait-Relationship von Wait MUSS auf den Wait-Prozessor selbst zurückgeschleift werden (Drag von Wait auf Wait, Relationship wait). Sonst verschwinden wartende FlowFiles. NUR expired und failure auto-terminieren, NICHT wait. 2. **~30s Geduld:** FlowFiles auf der wait-Schleife werden penalized (Default 30s). Nach dem Notify-Signal dauert die Freigabe daher bis zu 30s. Nicht zu früh "geht nicht" sagen. 3. **Signal-Reset zwischen Läufen:** Das Signal bleibt im MapCache. Für einen sauberen zweiten Lauf den MapCacheServer kurz deaktivieren und wieder aktivieren. 4. **ExecuteSQL-Stolperstein:** In NiFi 2.8 wirft ExecuteSQL beim Trigger ohne echte eingehende FlowFile teils IllegalStateException: Already added pointer to statements set. Für die reine Wait/Notify-Demo genügt als Job-B-Endknoten ein PutFile (schreibt eine Marker-Datei nach Freigabe) statt ExecuteSQL.

Diskussionsfragen: - Was passiert mit dem Signal nach dem ersten Durchlauf? (Stichwort: Signal-Verbrauch, Target Signal Count) - Wie würde man "B läuft genau einmal pro A-Lauf" sauber umsetzen? - Wozu dient die expired-Relationship von Wait (Timeout)?

Lab 3: Barriere / Fan-in: “Starte erst, wenn alle drei Dateien da sind”

Lernziel: Eine Synchronisationsbarriere bauen: ein Schritt startet erst, wenn mehrere Vorbedingungen erfüllt sind (mehrere Dateien angekommen).

Szenario: Drei Dateien (`fmea_entities.csv`, `sensordaten-kp300.csv`, `BTCUSDT_1D.csv`) sollen erst gemeinsam weiterverarbeitet werden, wenn alle drei vorliegen.

Variante A (einfach, über MergeContent):

```
GetFile —> MergeContent —merged—> PutFile (Sammelpaket)
```

- `MergeContent`: Minimum Number of Entries = 3, Maximum Number of Entries = 3, Max Bin Age = 30 sec (Sicherheitsventil, falls weniger als 3 kommen).
- Beobachtung: solange erst 1 oder 2 Dateien da sind, bleibt die Bin offen -> kein Output. Erst die dritte Datei löst `merged` aus.

Variante B (fortgeschritten, über Wait-Counter): - Jeder eingehende Datentyp ruft `Notify` mit `Signal Counter Name = eingang auf`. - `Wait` mit `Target Signal Count = 3` und demselben Counter lässt erst nach dem dritten Signal durch. - Vorteil gegenüber `MergeContent`: die Inhalte bleiben getrennte FlowFiles, nur die Freigabe ist synchronisiert.

Verifikation:

```
# einzeln einwerfen und Queue-Zähler beobachten:
cp ~/Knowledge/outputs/fmea_entities.csv      <Input>/
cp ~/Knowledge/outputs/sensordaten-kp300.csv  <Input>/
cp ~/Knowledge/outputs/btc-dd10-autoresearch-sandbox/BTCUSDT_1D.csv <Input>/
ls -l <Output-Pfad>/      # Sammelpaket erscheint erst nach der dritten Datei
```

Diskussionsfragen: - Was bewirkt `Max Bin Age`, und warum ist es ein wichtiges Sicherheitsventil? - Worin unterscheidet sich “warten auf 3 beliebige” von “warten auf 3 bestimmte” Dateien? (Hinweis: `Correlation Attribute Name`)

Lab 4: Zeitsteuerung + bedingte Verzweigung

Lernziel: Zeitliche Auslösung (Cron) und datenabhängige Verzweigung als zwei weitere Abhängigkeitsformen kennenlernen.

Szenario: Alle 30 Sekunden wird der Input-Ordner geprüft; je nach Dateiname wird in eine andere Zieltabelle geladen.

Teil A: Cron-/Timer-Auslösung - `GetFile` -> Tab **Scheduling**: `Scheduling Strategy = CRON driven`, z. B. `*/30 * * * * ?` (alle 30 Sekunden). Alternativ `Timer driven` mit `Run Schedule = 30 sec`. - Diskussion: Unterschied `Timer driven` (Intervall) vs. `CRON driven` (Uhrzeit/Kalender).

Teil B: Bedingte Verzweigung mit RouteOnAttribute

```
GetFile —> RouteOnAttribute —ist_fmea—> PutDatabaseRecord (Tabelle fmea_entities)
                               ↳sonst——> PutFile (Output/unsortiert)
```

- `RouteOnAttribute`: neue Property `ist_fmea` mit Wert `${filename:startsWith("fmea")}`
- Routing-Strategy = Route to Property name.
- `ist_fmea` -> `PutDatabaseRecord`; `unmatched` -> `PutFile`.

Verifikation:

```
cp ~/Knowledge/outputs/fmea_entities.csv      <Input>/    # -> landet in DB
cp ~/Knowledge/outputs/sensordaten-kp300.csv  <Input>/    # -> landet in Output/
unsortiert
ls -l <Output-Pfad>/unsortiert/
sqlite3 <DB> "SELECT COUNT(*) FROM fmea_entities;"
```

Diskussionsfragen: - Wann ist Cron sinnvoller als eine reine Datenauslösung (GetFile pollt ohnehin)? - Wie ließen sich mehrere Bedingungen (Dateiname und Größe) kombinieren?

Aufräumen / Reset zwischen den Übungen

```
# DB leeren (sonst summieren sich die INSERTs):
sqlite3 <DB> "DELETE FROM fmea_entities;"

# Arbeitsverzeichnisse leeren:
rm -f <Input>/* <Output>/* <Reports>/* <Archiv>/*

# Wait/Notify-Signale zurücksetzen: MapCacheServer kurz deaktivieren und wieder
aktivieren
# (oder NiFi-Prozessoren stoppen/starten).
```

Wichtig: GetFile-Polling-Intervall (~30s Geduld)

`GetFile` hat standardmäßig **Polling Interval = 30 sec**. Nach dem Start (oder nachdem du eine Datei einwirfst) kann es **bis zu 30 Sekunden** dauern, bis die Datei abgeholt wird. Im Test sah es kurz so aus, als “passiere nichts”: tatsächlich lief alles, nur eben erst beim nächsten Poll. Für die Schulung: nach dem Einwerfen ruhig **30-40s warten**, bevor man auf Fehlersuche geht. Wer es schneller mag, setzt in `GetFile` (Tab Properties) **Polling Interval** auf z.B. `2 sec`.

Häufige Stolpersteine

- **PutDatabaseRecord invalid:** `DBCPCConnectionPool` ist leer oder nicht enabled (URL + Driver Class + Driver Location prüfen).
- **Wait lässt nie durch:** `Notify` benutzt einen anderen `Release Signal Identifier` oder einen anderen Cache-Client als `Wait`. Beide müssen exakt übereinstimmen.
- **Doppelte Zeilen:** `GetFile` mit `Keep Source File = true` zieht dieselbe Datei mehrfach. Für saubere Tests `false` verwenden oder Datei nach Verarbeitung archivieren.
- **MergeContent gibt nichts aus:** Es liegen weniger Entries vor als `Minimum Number of Entries`, und `Max Bin Age` ist noch nicht erreicht.

Didaktischer Bogen (Vorschlag für den Trainer)

1. Lab 1 zeigt: Abhängigkeit ist erstmal nur eine Connection (`success`).
2. Lab 2 zeigt: echte Entkopplung zweier Jobs über Signale (Wait/Notify) = "B nach A".
3. Lab 3 zeigt: Synchronisationsbarriere (Fan-in) = "B nach mehreren A".
4. Lab 4 zeigt: zeit- und bedingungsgesteuerte Auslösung.

Roter Faden: In NiFi steuert man nicht "Jobs", sondern den **Fluss der FlowFiles** in die Eingangs-Queues.